

Concept de slice

Certains tableaux peuvent modifier leur taille: il s'agit des slices

Déclarer une variable

On peut déclarer une variable:

- En précisant son type
 - `var prenom string = "Alice"`
- Avec inférence de type
 - `var test = "Paris" // Sera un string`
- Déclaration courte (dans les fonctions)
 - `nom := "Bob"`
- Plusieurs variables à la fois
 - `x, y := 10, 20`
- On peut aussi échanger une valeur:
 - `x, y = y, x`

Surcharges

La surcharge ajoutant de la complexité au code et de la perte en compréhension, elle n'existe pas en Go.

Retour multiple

Étant donné que le `try-catch` n'existe pas en Go, la gestion des erreurs se fait grâce aux retours multiples.

Exemple:

```
func divide(a, b float64) (float64, error) {
    if b == 0 {
        return 0, errors.New("err")
    }
    return a/b, nil
}
res, err := divider(10, 2)
if err != nil {}
```

Fonctions variadic

Les fonctions variadics sont des fonctions qui peuvent prendre un nombre variable d'arguments.

Par exemple, les fonctions de formatage de string : `fmt.Printf`, `fmt.Println`

Closures

Les closures sont des fonctions qui se souviennent de l'environnement dans lequel elles ont été créées.

Structs

On peut définir, et initialiser un type struct:

```
// Définition du struct
type Produit struct {
    Nom string
    Prix float64
}

// 1. Par position
p1 := Produit{"iPhone 15", 999.99}

// 2. Par nom de champ (recommandé – ordre libre, lisible)
p2 := Produit{Nom: "iPhone 15", Prix: 999.99}

// 3. Valeur zéro puis assignation champ par champ
var p3 Produit
p3.Nom = "iPhone 15"
p3.Prix = 999.99
```

Struct tags

On peut aussi ajouter des tags aux structs afin qu'ils puissent être sérialisés en JSON:

```
type User struct {
    ID int `json:"id"`
    Name string `json:"name"`
    Email string `json:"email"`
    CreatedAt string `json:"created_at"`
}
```

Accessibilité

L'accessibilité est gérée grâce à la casse :

- Avec une majuscule, les fonctions et structs sont exportés.

- Avec une minuscule, les fonctions et structs restent interne.

defer

Le mot-clé `defer` permet de rajouter une instruction à la pile:

```
func test() {  
    fmt.Printf("1")  
    defer fmt.Printf("3")  
    fmt.Printf("2")  
}
```